

Battery Backtester: Software Design

Oct 1, 2026 · @Aditya Jain

Phases

The work is split into four phases. **Only Phase 1, the data foundation, is in scope now.** Everything about batteries, strategies and the simulator is a sketch for later and is kept further down for reference.

1. **Phase 1: Data foundation (now, about 2-3 weeks part-time).** Every IEX day-ahead and real-time 15-minute price and volume since April 2022, in one clean, point-in-time database, with a health report and a first look at daily price spreads.
 - Gate: the Phase 1 done criteria are met.
2. **Phase 2: Battery backtest on day-ahead prices.** Battery model, simple rules, perfect hindsight and a first forecast-driven strategy.
 - Gate: a first answer to whether forecasting beats a simple rule.
3. **Phase 3: Real-time market and better forecasts.** Two-stage day-ahead and real-time strategies; weather and demand data in the forecast.
4. **Phase 4: Extra revenue and products.** Grid services, the high-price segment, futures, and a live daily forecast.

Purpose and principles

The one question. On real Indian exchange prices, how much more does a smart, forecast-driven battery strategy earn than a simple "charge at noon, sell at dusk" rule? If the gap is large and steady, a battery-trading business has something to sell. If a simple rule captures nearly everything, it does not.

That question is answered in Phase 2. Phase 1 exists so the answer can be trusted: a backtest is only as good as the data under it.

Design principles.

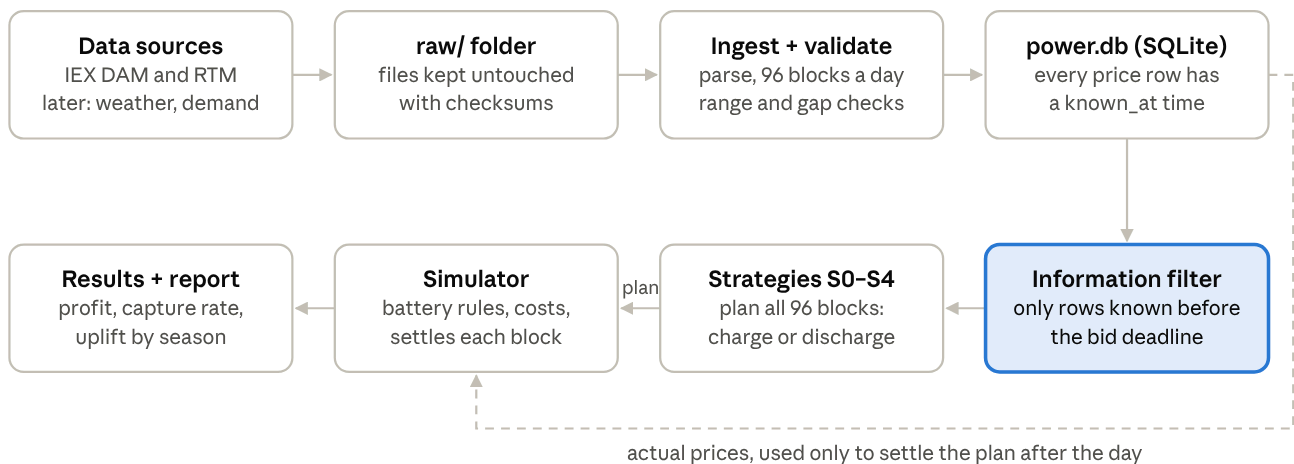
1. **One Python file, no frameworks.** Standard library plus pandas and numpy. Commands run from the terminal. Anyone can read the whole thing in an hour.
2. **Raw data is never thrown away.** Every downloaded file is kept as is; clean tables are rebuilt from it at any time.

3. **No look-ahead, enforced in code.** A strategy only ever sees data that existed at its decision time. For a day-ahead bid that means nothing after 12:00 the day before.
4. **Honest costs.** Losses, exchange fees, wear and price-cap limits are always on; the default is pessimistic.
5. **Measure against two anchors.** Every strategy is compared with the simplest rule (the floor) and with perfect hindsight (the ceiling). The useful number is where a strategy sits between them.
6. **Small, then wider.** Start with day-ahead prices only. Add the real-time market, weather and grid services one at a time, each only once the previous step works.

Architecture

Data flows one way: from exported files, through ingestion, into one database. The key design point is the split at the bottom: strategies get only filtered past data, while the simulator alone uses the actual delivery-day prices, and only to settle the plan afterwards.

Strategies see only the past; the simulator alone sees what actually happened



Backtester architecture · one Python file, one SQLite database

That split is what makes the results honest. If a strategy could ever see the delivery day's prices, its profit would be fiction.

Data sources

The first version needs only IEX's day-ahead and real-time market data. IEX's market snapshot pages offer 15-minute data with an export button, and the real-time market page states data is available from delivery day 1 April 2022 onward ([IEX RTM snapshot](#)). That is over four years of history.

Phase	Source	What it gives	Granularity	Used for
1	IEX day-ahead market snapshot	Buy bids, sell bids, cleared volume, clearing price	15-minute blocks, daily	The core price series
1	IEX real-time market snapshot	Same fields plus session and final scheduled volume	15-minute blocks, 48 sessions a day	Real-time prices and day-ahead vs real-time spreads
1 (optional)	IEX green and high-price day-ahead segments	Same fields, if the export format matches	15-minute blocks	Context; used from Phase 4
3	Weather archive and forecasts (a free historical weather API)	Temperature, humidity, cloud cover, wind	Hourly, by location	Price forecasting
3	Grid-India daily reports	National and regional demand, generation mix	Daily or hourly	Price forecasting
4	IEX grid-service market data	Reserve market prices	15-minute blocks	Extra revenue streams
4	MCX futures settlement prices	Monthly futures prices	Daily	Hedging tests

How to collect phase 1 data. Start with manual monthly exports from the IEX pages, saved into a raw folder. That is about 54 files per market for the full history, a few hours of work. Automate downloading only after checking IEX's terms of use, and keep requests slow and polite.

What to record about each file. Its source page, delivery date range, the date and time it was downloaded, and a checksum. If IEX later revises a number, the history shows both versions.

Data ingestion and storage

Ingestion turns raw exported files into one clean table, with one row per market per 15-minute block. Raw files stay untouched; the clean table can be rebuilt from them at any time.

Storage. A single SQLite file (`power.db`) plus a `raw/` folder. SQLite needs no server, is one file you can copy, and comfortably holds decades of 15-minute data.

Tables.

Table	One row per	Key columns
<code>files</code>	Downloaded raw file	path, source, market, first and last delivery date, downloaded_at, checksum
<code>blocks</code>	Market, delivery date and block	market (DAM, RTM), delivery_date, block (1-96), buy_bid_mw, sell_bid_mw, mcv_mw, mcp_rs_mwh, file_id, known_at
<code>runs</code>	One backtest run	run_id, strategy, battery settings, date range, created_at, code version
<code>results</code>	Run and day	run_id, delivery_date, energy bought and sold, revenue, costs, profit, cycles

The `known_at` column is the point-in-time rule. Each price row records when that price became public. Day-ahead prices for date D are known around 13:00–14:30 on D–1; a real-time block is known shortly after its session clears. Strategies may only read rows whose `known_at` is before their decision time.

Ingestion steps.

1. Read each new file in `raw/` and register it in `files` with a checksum, so the same file is never loaded twice.
2. Parse it into rows; standardise dates, block numbers (1–96) and units (₹/MWh, MW).
3. Write rows to `blocks`. If a row already exists with different values, keep both and flag it, rather than overwriting.

4. Run the validation checks and print a short report.

Validation checks, run after every ingest.

- Every delivery day has exactly 96 blocks per market.
- Prices are between ₹0 and the cap in force on that date (₹10,000/MWh for normal markets since April 2023).
- Cleared volume is never more than buy bids or sell bids.
- No missing days in the requested range; gaps are listed.
- A known day matches published numbers, for example the 29 September 2026 day-ahead average of ₹7,448.55/MWh.

Phase 1 in detail: the data foundation

Goal. A clean, trustworthy, point-in-time database of every IEX day-ahead and real-time 15-minute price and volume since April 2022, that can be refreshed in minutes and checked at a glance. No battery logic, no forecasts.

In and out of scope.

In Phase 1	Not in Phase 1
IEX day-ahead and real-time data, April 2022 onward	Battery model, strategies, simulator
Green and high-price day-ahead segments, if the export format matches	Weather, Grid-India demand, futures data
File registry, <code>known_at</code> times, revision handling	Price forecasts
Validation, health report, first look at spreads	Automated downloading, until IEX's terms are checked
A routine to add new days	Any web dashboard

Tables for Phase 1. `files` and `blocks` as described above, plus an `issues` table that records every validation finding (date, market, block, check, detail) so problems are listed rather than silently dropped. A `blocks_latest` view shows the most recent version of each block when a file has been re-downloaded.

Columns in `blocks` . market, delivery_date, block (1-96), block_start, purchase_bid_mw, sell_bid_mw, mcv_mw, final_scheduled_volume_mw (real-time only), mcp_rs_mwh, session_id (real-time only), file_id, known_at, loaded_at.

known_at rules, set conservatively. Day-ahead prices for delivery day D: 15:00 on D-1, after results are normally published. Real-time prices: the start of the delivery block. Both are cautious on purpose; check against actual publication times and tighten later if needed.

Tasks, in order.

1. **Inspect one export.** Download one month of day-ahead and real-time data by hand. Note the file type, header rows, column names, units, how blocks and times are written, and how midnight is handled. Write these down as a short "file contract" for each market.
2. **Create the folders and database.** `raw/dam/` , `raw/rtm/` , and `power.db` with the `files` , `blocks` and `issues` tables.
3. **Write one parser per market.** It turns one file into rows and stops with a clear error if the columns differ from the file contract.
4. **Register files and avoid duplicates.** Store a checksum per file; skip files already loaded; if a re-downloaded file differs, load it as a new version rather than overwriting.
5. **Set `known_at`** for every row using the rules above.
6. **Run validation** after every load and write findings to `issues` .
7. **Load the full history:** about 54 monthly files per market, April 2022 to September 2026.
8. **Build the health report:** a coverage calendar of present and missing days per market, issue counts by type, yearly price ranges, the share of blocks at the price cap, and the 29 September 2026 check.
9. **Build the first look:** for every day, the lowest and highest block price, the gap between them, and the best two-hour buy window versus the best later two-hour sell window; monthly averages of each. This gives an early read on how large and steady spreads are, before any battery code exists.
10. **Write the refresh routine:** export the latest month, run ingest and validate, and check the health report. Document it at the top of the file.

Phase 1 commands.

```
python bess.py init                # create folders and database
python bess.py ingest raw/         # load new or changed files
python bess.py validate            # run checks, write issues
python bess.py health              # write reports/health.html
python bess.py firstlook           # write reports/firstlook.csv and a
chart                               chart
python bess.py export --market DAM --from 2025-04-01 --to 2026-03-31  # CSV
for your own analysis
```

Done criteria.

- ☐ Every day from April 2022 to September 2026 is present for both markets, or listed as missing with a reason.
- ☐ No unexplained validation failures.
- ☐ The 29 September 2026 day-ahead average reproduces ₹7,448.55/MWh.
- ☐ Running ingest twice on the same files changes nothing.
- ☐ The health report and first-look file are produced from one command each.
- ☐ Adding a new month takes under 15 minutes from export to checked data.

Later phases: sketch only, not for now

The sections below describe Phases 2–4. They are kept so the Phase 1 database is designed with them in mind, but nothing here is built until Phase 1 is done.

The battery model (Phase 2)

The simulator represents one battery as a handful of settings and enforces the physical and cost rules on every 15-minute block. All settings are inputs, so the same code can test a 2-hour or 4-hour battery, cheap or expensive cells.

Settings.

Setting	Meaning	Starting value (to be checked)
Power	Maximum charge or discharge rate	100 MW
Energy	Usable capacity	200 MWh (a 2-hour battery)

Setting	Meaning	Starting value (to be checked)
Round-trip efficiency	Share of bought energy that comes back	87% (split evenly between charging and discharging)
State of charge band	Allowed range of fullness	10–90%
Starting state of charge	Fullness at the start of each day	10%, and the day must end at the same level
Cycle limit	Maximum full cycles per day	1 at first; test 2 later
Wear cost	Cost charged per MWh discharged, standing in for degradation	Set from cell cost and warranted cycles; test several values
Exchange fee	Charged on both buying and selling	2 paise per kWh per side
Transmission charges and losses	Extra cost per MWh bought or sold	Zero for a waiver case; positive for a standalone case
Participation limit	Largest share of a block's cleared volume the battery may take	10% of cleared volume (see simulator)

Rules the simulator enforces in every block.

1. Charge and discharge cannot happen in the same block.
2. Energy per block is power $\times$ 0.25 hours.
3. State of charge after charging rises by the energy bought $\times$ charging efficiency; after discharging it falls by the energy sold $\div$ discharging efficiency.
4. State of charge stays inside its band; any instruction that breaks it is clipped and the clipping is logged.
5. Every MWh bought pays the clearing price plus fees and charges; every MWh sold earns the clearing price minus fees, charges and wear cost.

Deliberately left out of version 1: deviation penalties (the battery is assumed to follow its schedule exactly), grid-service income, and capacity fading over the years. Each is a later addition.

Strategies to compare (Phases 2–3)

Five strategies, from dumbest to smartest, all run on the same battery and the same days. The business case lives in the gap between the simple rules and the forecast-driven ones.

#	Strategy	How it decides	Role
S0	Fixed windows	Charge 11:00–13:00, discharge 18:00–20:00, every day	The floor: what anyone can do with no software
S1	Copy the last known day	Find the cheapest and dearest windows in the most recent published day-ahead prices, and use them	A smarter rule of thumb
S2	Forecast plus optimiser	Forecast tomorrow's 96 prices, then find the best charge and discharge plan for that forecast	The product being tested
S3	Perfect hindsight	Run the same optimiser on the actual prices	The ceiling: no strategy can beat it
S4	Day-ahead plus real-time	Commit S2's plan in the day-ahead market, then re-plan before each real-time session and trade the changes	Tests whether the real-time market adds value

The forecast in S2 starts simple and gets better. Version 1 uses an average of the same weekday over the last four weeks, blended with the most recent known day. Later versions add weather, demand and season. Each version is a separate strategy, so improvement is measured, not assumed.

The optimiser. For a given price forecast, it searches every possible path of state of charge through the 96 blocks and keeps the most profitable one, using dynamic programming over state of charge in 1% steps. It needs no external solver, runs in well under a second per day, and is easy to read and check.

How bids are treated in version 1. The battery buys and sells its planned quantities at whatever the clearing price turns out to be, and is assumed too small to move that price. When a forecast is wrong, it may buy in a block that turns out expensive. Version 2 adds price limits (buy only below a price, sell only above one) to test whether they protect against bad forecasts.

The simulator (Phase 2)

The simulator replays history one delivery day at a time, exactly as a live battery would have experienced it. It asks the strategy for a plan using only information available at the decision time, then settles that plan against the real prices.

One simulated day, step by step.

1. **Set the clock** to the day-ahead bid deadline: 12:00 on the day before delivery.
2. **Build the information set:** all price rows whose `known_at` is before that time. Nothing from the delivery day itself is included.
3. **Ask the strategy for a plan:** a charge or discharge quantity for each of the 96 blocks.
4. **Check the plan against the battery rules:** power limits, state of charge band, cycle limit. Clip and log anything that breaks them.
5. **Apply the participation limit:** in each block, the battery may not buy or sell more than a set share of the block's cleared volume. This matters most in the evening, when on some days only about 1 GW is offered across India.
6. **Settle** each block at the actual clearing price, applying efficiency, fees, charges and wear cost.
7. **For S4 only**, repeat steps 1–6 before each real-time session with the newer information, trading only the change from the existing plan at real-time prices.
8. **Record** the day's result and carry the ending state of charge into the next day.

Look-ahead protection, built into the code. Strategies never query the database directly. They receive only the filtered information set from step 2. A test feeds a strategy deliberately corrupted future data; if its decisions change, the test fails.

Price impact. Public data gives only total buy and sell bids per block, not the full bid curves, so the simulator cannot compute exactly how a battery would move the price. The participation limit is a simple stand-in. Results for large batteries should be read with that caveat.

Outputs and the go/no-go metrics (Phase 2)

Each run produces a daily results table and a one-page report. The decision rests on three numbers: how much of the hindsight ceiling each strategy captures, how much the forecast-driven strategy beats the simple rules by, and how steady that gap is across seasons.

Metrics, per strategy.

Metric	What it tells you
Profit per MW per year	The headline earnings, comparable with lease tariffs of ₹2.3–2.9 lakh per MW per month
Capture rate	Profit ÷ perfect-hindsight profit; how close to the ceiling a strategy gets
Uplift over S0 and S1	Extra profit from the forecast-driven strategy, in rupees and per cent
Uplift by month and season	Whether the gain is steady or comes from a few unusual days
Share of days S2 beats S1	How reliable the edge is day to day
Worst month	How bad it gets when forecasts go wrong
Cycles and clipped instructions	Whether the strategy respects the battery's limits

The report. One HTML page per run, with monthly profit by strategy, capture rate by season, and a table of the 20 best and worst days for S2 against S1. The daily table is also saved as CSV for further analysis.

Sanity checks before trusting any result.

- Perfect hindsight profit is in the same range as published estimates, such as Ember's annual revenue estimate for two-hour merchant batteries.
- Doubling the wear cost reduces cycling and profit, never increases it.
- Removing a month of data changes results only for that month.

The decision rule. Go ahead if S2 (or S4) captures clearly more of the ceiling than S1 in every season, and the extra rupees per MW are large enough that an owner would share part of them. The size of "large enough" comes from the owner interviews, not from this software.

File layout, commands and build order

One Python file holds everything; data and reports sit in plain folders beside it.

```

bess/
  bess.py          # everything: config, database, ingest, validate,
battery, strategies, simulator, report, commands
  power.db         # SQLite database, rebuilt from raw/ at any time
  raw/
    dam/           # IEX day-ahead exports, untouched
    rtm/           # IEX real-time exports, untouched
  reports/
    <run_id>/      # report.html and daily.csv for each run

```

Sections inside `bess.py` , top to bottom: settings, database, ingest, validation, battery rules, forecasts, optimiser, strategies, simulator, report, self-tests, command line.

Commands.

```

python bess.py ingest raw/
python bess.py validate --from 2022-04-01 --to 2026-09-30
python bess.py backtest --strategies S0,S1,S2,S3 --from 2022-04-01 --to 2026-
09-30 --power 100 --energy 200
python bess.py report <run_id>
python bess.py selftest

```

Dependencies: Python 3.11 or later, pandas, numpy, openpyxl (to read Excel exports) and matplotlib (for report charts).

Build order.

1. **Phase 1 (now):** follow the ten tasks in the Phase 1 section; about 2–3 weeks part-time.
2. **Phase 2:** battery rules, settlement and optimiser; S0, S1, S3 and a simple S2; the first report.
3. **Phase 3:** real-time data in strategies (S4); weather and demand in the forecast.
4. **Phase 4:** grid services, high-price segment, futures, live daily forecasts.

Risks and open questions

- **Data rights.** Check IEX's terms of use before automating downloads; manual exports are the safe start. Export formats may change without notice, so the parser should fail loudly.
- **Price impact.** Without full bid curves, the simulator cannot model how a large battery moves prices. The participation limit is a rough stand-in.

- **Battery parameters.** Efficiency and wear cost are estimates. Report results across a range rather than a single number.
- **Rule changes inside the history.** The price cap fell from ₹12 to ₹10 in April 2023, and deviation rules changed in 2024. Results should be split by period, and the latest period weighted most.
- **Missing revenue.** Grid services, about 15–20% of merchant revenue by Ember's estimate, are left out of version 1. The backtest therefore understates total earnings, though not necessarily the uplift.
- **The future may differ from the past.** Market coupling, more batteries narrowing the spread, and possible grid-charging restrictions are not in the historical data.
- **Open question for interviews:** which battery size, duration and contract type do likely first customers actually have? Run the backtest for those settings first.